

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11. ES6 JS IV: 擴展

11. ES6 JS IV: 擴展 學習目標

- 理解陣列、物件和字串的擴展方法
- 關於JS的每一課的回顧

11. ES6 JS IV: 擴展 內容

11.1

複習上節課

11.3

物件擴展

11.5

回顧

11.2

陣列擴展

11.4

字串擴展

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11.1 複習上節課

11. ES6 JS IV: 擴展 模板文字

- 在ES6之前，字串可以用單引號或雙引號來包住
- 需要更多的程式碼來實現更多的字串功能
- 模板文字使用反斜線(`) 來包住字串
- 模板文字可以包住多行字串，並會記錄字串中的新行
- 單引號和雙引號也可以在沒有跳脫字元的情況下使用

11. ES6 JS IV: 擴展 模板文字

- 在ES6之前，將變數傳入字串需要使用運算符，這會使簡單的句子變得冗長
- 模板文字支援在字串內部傳遞變數，帶來更多的靈活性和更短的程式碼
- 在字串`${variable}`裡有特殊的區塊語法

11. ES6 JS IV: 擴展 陣列解構

- 陣列解構提供更快捷的方式，將變數分配給可迭代物件中的項目
- 在ES6之前，必須為陣列中的每個項目使用索引，這既耗時又會產生大量的程式碼

```
function getAge(...ages) {  
    return ages;  
}  
  
let [x, y, z] = getAge(10,20,30);  
console.log(x); //10
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展 陣列解構

- 如果函式返回的陣列項目少於變數，剩下的變數將是未定義的
- 如果函式返回的陣列項目多於變數，所有的變數將被依次分配，剩下的陣列項目則保持不變
- 假如有太多的陣列項目，可以用剩餘語法(Rest Syntax) “...” 來為剩下的陣列項目分配給指定的名稱
- 假如有未定義的變數，可以用默認參數(Default Parameters)來為變數設置默認值

11. ES6 JS IV: 擴展 陣列解構

- 陣列解構可以用兩種方式處理巢狀陣列:
- 為整個巢狀陣列指定一個變數
- 或者為巢狀陣列的每個項目分配一個變數
- 陣列的解構可以用來交換值，這使得這項工作比ES6之前要簡單得多

11. ES6 JS IV: 擴展 箭頭函式

- 箭頭函式加快了編碼過程，並簡化程式碼
- 通過使用箭頭前綴=>，可以在一行內表示一個函式區塊
- 除非該變數是函式主體中唯一使用的變數，否則需要包含變數的括號
- 如果在換行後使用箭頭，會導致語法錯誤

11. ES6 JS IV: 擴展 ES6模組

- 模組是JS腳本的擴展，支援使用其他JS文件中的函式和方法

```
<div id="display"></div>
<script>
  // Assuming this is on the message.js file
  export let message = "This is a message"

  import { message } from './message.js'
  let div = document.querySelector("#display");
  div.textContent = message; // message will be shown on
  // div despite inside another file
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11.2 陣列擴展

11. ES6 JS IV: 擴展 陣列擴展

- 在ES6之前，向陣列構造函式傳遞一個數字時，
- JavaScript會建立一個長度為3的陣列
- 這時候，陣列的元素是未定義的，因為只指定了陣列的長度

```
let arr = new Array(3);  
console.log(arr);  
console.log(arr[0]);
```

```
▼ (3) [empty × 3] ⓘ  
  length: 3  
  ▶ __proto__: Array(0)  
undefined
```

11. ES6 JS IV: 擴展 陣列擴展

- 然而，向陣列構造函式傳遞一個不是數字的值時，JS會建立一個帶有這個值的陣列
- 有時這會導致問題和錯誤，尤其是當不清楚傳入的數據類型時

```
let arr = new Array(3);  
console.log(arr);  
console.log(arr[0]);
```

```
▶ ["3"]  
3
```

11. ES6 JS IV: 擴展 陣列擴展

- **Array.of()**是一個類似於陣列構造函式的方法，會建立一個包含傳入的值的陣列
- 無論什麼類型元素的，都會建立與傳入該方法的參數數量相等的元素
- 注意**Array.of()**方法不是一個構造函式，不可使用**new**關鍵字

```
let arr = Array.of(3);  
console.log(arr); // [3]  
console.log(arr[0]); // 3
```

11. ES6 JS IV: 擴展 陣列擴展

- `Array.from()`從傳入的陣列中建立一個新的陣列實例(instance)
- 在字串的情況下，字串的每個字元都被轉換為新陣列的元素

```
let arr = Array.from("ABCDEFGH");  
console.log(arr);
```

```
► (7) ["A", "B", "C", "D", "F", "G", "H"]
```


11. ES6 JS IV: 擴展 陣列擴展活動

- 活動：使用Array.from()返回新的陣列，裡面的所有值都是平方
- 建立數字陣列
- 使用Array.from()，應用回調函式，將每個元素乘以自身(x代表陣列中的每個元素)
- 顯示結果

```
let numlist = [1, 2, 3, 4, 5, 6, 7, 7, 8, 9, 10]
let result = Array.from(numlist, x => x * x);
console.log(result);
```

11. ES6 JS IV: 擴展 陣列擴展

- `Array.find()`是一個函式，在給定的條件下，找到陣列中的第一個有效元素
- 下面的例子，是一個包含10個人的年齡的陣列
- 如何找到第一個年齡超過30歲的人？

```
let agelist = [18, 18, 21, 26, 28, 31, 35, 38, 45, 52];
```

11. ES6 JS IV: 擴展 陣列擴展

- 與Array.from()類似，Array.find()接受一個帶有條件的回調函式，該條件將被應用於陣列中的每個元素
- 元素只有當滿足回調函式中所述的條件時，才有效

```
let agelist = [18, 18, 21, 26, 28, 31, 35, 38, 45, 52];  
let above30 = agelist.find(x => x > 30);  
console.log(above30);
```

31

11. ES6 JS IV: 擴展 陣列擴展

- `Array.findIndex()`與`Array.find()`功能非常相似
- 被作用在一個已存在的陣列上，並接受一個有條件的回調函式作為參數
- 第一個有效元素的索引將被返回

```
let agelist = [18, 18, 21, 26, 28, 31, 35, 38, 45, 52];  
let above30 = agelist.findIndex(x => x > 30);  
console.log(above30);
```

5

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11.3 物件擴展

11. ES6 JS IV: 擴展物件擴展

- `Object.assign()`將物件的所有屬性分配給目標物件
- 它需要2個參數:
- `targetObject` - 要被分配屬性的目標物件
- `originalObject` - 複製屬性的來源物件

```
let name = {  
  name: "John"  
};  
  
let age = {  
  age: 27  
};  
  
let person = Object.assign(name, age);  
console.log(person);
```

```
▼ {name: "John", age: 27} ⓘ  
  age: 27  
  name: "John"  
  ► __proto__: Object
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有 · 翻印必究

11. ES6 JS IV: 擴展物件擴展

- 調用Object.assign時，在targetObject參數上提供一個空物件，可以用來複製物件
- 也可以將多個物件(originalObjects)合併在一起

```
let person = {  
  name: "John",  
  age: 27,  
  title: "Salesman"  
};  
  
let clonedperson = Object.assign({}, person);  
console.log(clonedperson);
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展物件擴展

- **Object.is()**是一個不被限制用於物件的方法
- 運作原理類似於===運算符，但有兩個不同之處：
- -0和+0
- NaN
- 在大多數情況下，Object.is()不被使用，因為使用===運算符更簡單，而且使用情況很少

11. ES6 JS IV: 擴展物件擴展

- `===`運算符將`-0`和`+0`視為相同的值
- `Object.is()`則視為不同的值處理
- 使用`===`運算符，`NaN`視為不等於`NaN`
- `Object.is()`則視為相同的值

```
console.log(-0 === +0); // true
console.log(Object.is(-0, +0)); // false

console.log(NaN === NaN); // false
console.log(Object.is(NaN, NaN)); // true
```

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11.4 字串擴展

11. ES6 JS IV: 擴展 字串擴展

- ES6為字串字頭增加了新的方法，使其更容易被操作
- `string.startsWith()`函式檢查目標字串是否以該搜索字串開頭
- 該方法是由要搜索的目標字串調用的
- 該函式需要2個參數：搜索字串，以及要搜索的起始位置(默認值是0)

```
let hello = "Welcome to JS Tutorial!"  
console.log(hello.startsWith("Welcome")); // true
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展 字串擴展

- `startsWith()`方法是區分大小寫的，所以要確保與目標字串的開始位置完全匹配
- 位置告訴方法從哪裡開始，在下面第一個例子中，開始位置被放在 `welcome` 之後

```
let hello = "Welcome to JS Tutorial!"  
console.log(hello.startsWith("welcome")); // false  
  
console.log(hello.startsWith("to", 8)); // starts from the letter t
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展 字串擴展

- `endsWith()`的運作原理與`startsWith()`類似，但在目標字串的末尾處檢查是否以該搜索字串結尾
- 該函式也需要2個參數：搜索字串，以及目標字串的結束位置

```
let hello = "Welcome to JS Tutorial!"  
console.log(hello.endsWith("Tutorial!")); // true
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展 字串擴展

- `endsWith()`也是區分大小寫的
- 結束位置決定了目標字串在哪裏完結，在下面的例子中，結束位置被放在 `Tutorial` 之前
- 字串結束在指定的索引之前，搜索字串"JS"在索引 11 和 12，所以位置必須放在 13，使搜索結果為真

```
console.log(hello.endsWith("JS", 13)); //ends at the letter S, before the white space
```

11. ES6 JS IV: 擴展 字串擴展

- `string.includes()`方法是ES6增加的另一個方法
- 檢查目標字串中是否包含搜索字串
- 該函式也需要兩個參數：搜索字串和起始位置

```
let hello = "Welcome to JS Tutorial!"  
console.log(hello.includes("to")); // true
```

11. ES6 JS IV: 擴展 字串擴展

- 與其他字串方法類似，`string.include()`是區分大小寫的
- 位置決定了搜索的開始

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

11.5 回顧

ES6語法

- ES6是JavaScript的編程語言標準，使其更加現代化和可讀
- `let`可以用來宣告新的變數，就像`var`一樣，但`let`是有區塊範圍的
- 有區塊範圍的變數在非同步環境中運行良好
- 用`let`宣告的變數不能在其區塊範圍外被引用
- `let`不能重新宣告同一個變數
- `let`宣告的變數不能在宣告前被引用，也不能被提升(hoist)

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展

ES6語法

- `const`關鍵字的作用與`let`關鍵字類似，只是建立的變數是唯讀的
- 不能把常數重新賦值，但是可以重新賦值給常數中的屬性，因為不會改變常數本身
- 如果該參數上沒有傳遞任何值或未定義的話，默認參數允許一個函式的參數默認為某個值

11. ES6 JS IV: 擴展

ES6語法

- 剩餘參數的前綴是“...”，以陣列的形式表示剩餘語法後的任何數量的參數
- 剩餘參數只能作為一個函式的最後一個參數使用
- 展開運算符的前綴亦是“...”，被用來解構陣列內的元素
- 展開運算符可以在陣列內的任何地方使用

ES6語法

- 如果物件的屬性名稱和值是相同的，那麼只需要使用其中一組即可建立物件
- 在一個物件的屬性名稱處使用方括號，括號內的所有內容將作為字串處理
- 如果是一個變數，將使用該變數的值並作為字串處理
- 在ES6，如果物件內的函式的屬性名與函式名相同，則不需要包含該屬性名

ES6語法

- **For...of**是一種對可迭代物件進行遍歷的方法，運作原理與**for迴圈(for loop)**非常相似，但程式碼要少得多
- **array.entries()**可以與**for...of**一起使用，以獲得陣列中元素的索引和值
- 八進制文字有一個前綴“0o”（0後面是字母o），後面是八進制數字
- 二進制文字有一個前綴“0b”，後面是二進制數字
- 模板文字是由反斜線(`)定義的，支援在字串中插入變數，也允許在字串中插入單引號和雙引號

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

11. ES6 JS IV: 擴展 陣列解構

- 解構賦值支援使用陣列同時賦值多個變數
- 剩餘語法可以用在陣列解構上，將所有剩餘的值作為一個陣列，並將其分配給一個變數
- 如果陣列解構包含一個未定義的值，可以使用默認參數為變數分配默認值
- 如果變數的格式與值完全相同，巢狀陣列的解構也是可能的

11. ES6 JS IV: 擴展 箭頭函式

- 箭頭函式的前綴是一個箭頭=>，可以加快函式的建立速度
- 包含參數的括號即使在箭頭前沒有參數也應該使用，除非參數是函式內部使用的唯一變數
- 如果在箭頭前綴之後使用區塊語法，則必須使用返回(return)關鍵字
- 箭頭函式中不允許換行，除非箭頭前綴放在換行之前
- 模組(modules)是JS腳本的擴展，支援導入和導出其他方法、變數來使用

11. ES6 JS IV: 擴展 作業

- 按照作業文件的要求，在課堂上完成

11. ES6 JS IV: 擴展 參考

- 如果需要更多的解釋，請使用:
- <https://www.javascripttutorial.net/es6/>
- <https://javascript.info/>
- 如果需要更具體的答案，請使用:
- <https://stackoverflow.com/>